

Kaylie Zhao

Email kzhao051@ucr.edu

Date Feb-13-2026

In completing this assignment, I consulted the following:

- Lecture slides and videos provided by the professor
- The project 1 sample report from this class
- Python 3 standard library: <https://docs.python.org/3/library/index.html>
- Python nodes: https://www.tutorialspoint.com/python/python_nodes.htm
- Heapq documentation: <https://docs.python.org/3/library/heapq.html>
- 8 Puzzle visual: <https://www.youtube.com/watch?v=dvWk0vgHij5>
- Python Sets:
<https://www.dataquest.io/blog/tutorial-everything-you-need-to-know-about-python-sets/#:~:text=Note%20that%20while%20a%20Python,floats%2C%20tuples%2C%20or%20strings>
- Heapq subroutines for node handling
- Built-in types Python documentation:
<https://docs.python.org/3.6/library/stdtypes.html>
- Sliding Puzzle Wikipedia: https://en.wikipedia.org/wiki/Sliding_puzzle
- Geeks for Geeks 8 puzzle searches:
<https://www.geeksforgeeks.org/artificial-intelligence/8-puzzle-problem-in-ai/>

All important code is original. Unimportant subroutines that are not completely original are: heapq subroutines for node handling and general search pseudocode

Outline of this report:

- Cover page: (this page)
- My report: Pages 2 to 6
- Sample trace on an easy problem, page 7
- Sample trace on hard problem pages 8 to 9
- My code page is listed on page 9 in a GitHub link. If you want to run my code, here is a URL to the github that runs the 8-puzzle search.
https://github.com/kayliezhao/cs170_lab_1

CS170: Project 1: The Eight-Puzzle

Kaylie Zhao, Feb-13-2026

Introduction

The sliding tile puzzle problem is a puzzle that challenges players to slide a board piece among different routes to get to the goal state (Wikipedia). More specifically, the 8-puzzle is a 3-by-3 puzzle that has 8 square pieces on a grid with a blank or empty tile. The 8-puzzle is numbered accordingly from 1 to 8, with the last tile slot being empty (Wikipedia). There are many other variations of this puzzle with different grid sizes like the 3 puzzle (2 x 2 grid), or even the 15-puzzle (4 x 4 grid). The goal is to make a series of moves to reach the goal state of numerical order, with the blank tile being at the bottom-rightmost spot.



Figure 1: A picture of an 8-puzzle game I found online at <https://sliding.toys/mystic-square/8-puzzle/> to assist me with this project.

Dr. Eamonn Keogh tasked his class of CS 170 Introduction to Artificial Intelligence at the University of California, Riverside, during the Winter of 2026 to create this project on the 8-Puzzle. This report will be a result of all my findings during the making of this project. It goes over Uniform Cost Search, A* with the Misplaced Tile Heuristic, and A* with the Manhattan Distance Heuristic. The report will also show sample traces of select difficulties and the code for the project. The tech stack used for this project was Python3 for coding, GitHub for the repository and codespace, and Google Sheets for the visualizations of graphs. This project also showcases the solution depth, number of nodes expanded, and max queue size. The solution depth is the

number of moves it'll take to solve the puzzle. The number of nodes expanded is the number of states in the puzzle that my program had to look through before finding the solution. The max queue size is the maximum number of puzzle states my program had at any time during its search.

Comparison of Algorithms

Three algorithms were implemented for the purpose of this project. There's Uniform Cost Search, A* with the Misplaced Tile Heuristic, and A* with the Manhattan Distance Heuristic.

Uniform Cost Search

As stated in the sample project provided by Dr. Eamonn Keogh, **Uniform Cost Search** "is just A* with $h(n)$ hardcoded to equal zero." This essentially means that the heuristic is ignored, so now it's $f(n) = g(n) + 0$, so the nodes will just expand to the cheapest node available. $f(n)$ represents the total estimated cost of the cheapest solution, $g(n)$ represents the actual cost to reach the current state, and $h(n)$ represents the heuristic. I found that Uniform Cost Search here highly resembles breadth-first search with the way it expands before moving onto the next depth level.

A* Misplaced Tile Heuristic



$h(n) = 2$ misplaced tiles

Figure 2: An example from my project of the misplaced tile Heuristic. This shows the start state and the goal state of the puzzle.

A* with **Misplaced Tile Heuristic** was the second algorithm implemented. The 0 is a placeholder for the blank tile, $h(n)$ represents the number of misplaced tiles, and $g(n)$ is the number of moves already made. Figure 2 shows $h(n) = 2$, as there are two misplaced tiles, 3 and 6, as shown boxed in red. The node priority in this algorithm is represented by $f(n) = g(n) + h(n)$. This shows a more reliable and efficient algorithm than Uniform Cost Search since it prioritizes the nodes with a cheaper solution ($f(n)$).

A* Manhattan Distance Heuristic

The A* with **Manhattan Distance Heuristic** is the third algorithm implemented. Like the Misplaced Tile Heuristic, the Manhattan Distance Heuristic also considers the cost of future expansions and uses the A* to calculate the $f(n) = g(n) + h(n)$ (Geeks for Geeks). The $h(n)$ is the sum of the distances of all the tiles from the current position to the goal state. This method is more efficient than the previous algorithms in terms of caring about the distance of the tile from the correct location. It's defined as $h(n) = \text{horizontal} + \text{vertical distance (slides)}$.

Comparison of Algorithms on Sample Puzzles

Depth 0	Depth 2	Depth 4	Depth 8	Depth 12	Depth 16	Depth 20	Depth 24
123 456 780	123 456 078	123 506 478	136 502 478	136 507 482	167 503 482	712 485 630	072 461 358

Figure 3: Test cases from the project sample document.

To test the performance and conduct a data analysis on the different algorithms, I used the test cases as shown in Figure 3. These test cases were taken from the project 1 sample report document and used solely for testing purposes. I ran each depth through each algorithm and collected the data onto a Google Sheet for data analysis.

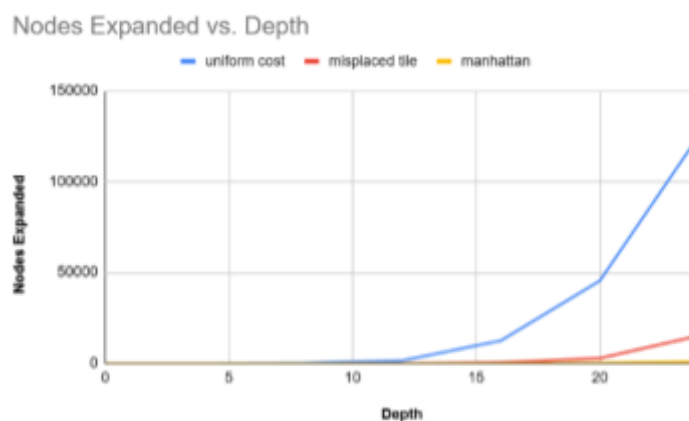


Figure 4: Line chart of Nodes Expanded (y) vs. Solution Depth (x) for Uniform Cost Search, A* with Misplaced Tile Heuristic, and A* with Manhattan Distance Heuristic

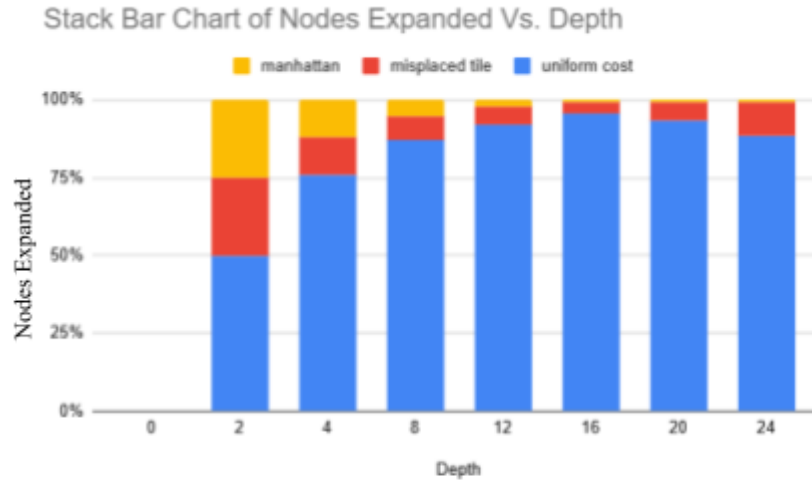


Figure 5: Stacked Bar Chart of Nodes Expanded (y) vs. Solution Depth (x) for Uniform Cost Search, A* with Misplaced Tile Heuristic, and A* with Manhattan Distance Heuristic

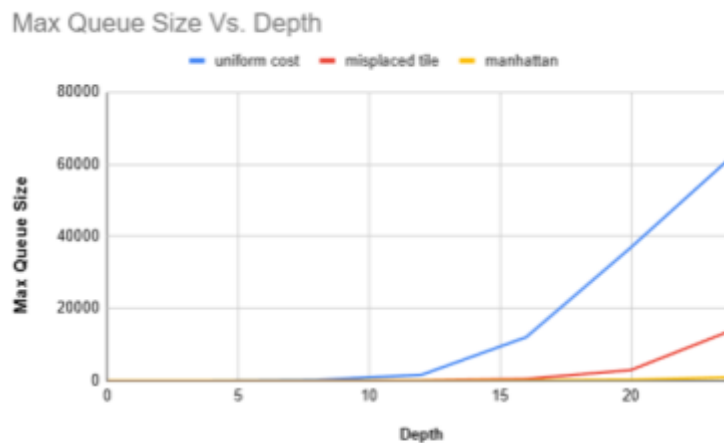


Figure 6: Line chart of Max Queue Size (y) vs. Solution Depth (x) for Uniform Cost Search, A* with Misplaced Tile Heuristic, and A* with Manhattan Distance Heuristic

By using the test cases of the depths in Figure 3, I created line charts and a stacked bar graph as shown in Figures 4, 5, and 6. Figures 4 and 5 both show a comparison between the number of nodes expanded and the solution depth for each search algorithm. Figure 6 clearly shows a comparison between the max queue size and the depth for each search algorithm.

As shown in Figures 4 and 5, the Uniform Cost Search algorithm explores the most nodes as the depth of the puzzle increases. Although it may appear as if A* with Misplaced Tile Heuristic and A* with Manhattan Distance Heuristic (MDH) are the same in terms of the nodes explored versus depth on the lower levels, the deeper the depth goes, the more they

start to distinguish themselves from each other. Figure 6 shows that Uniform Cost Search's max queue size increases greatly the deeper the depth goes. Misplaced Tile Heuristic is on the lower side, but the MDH is the lowest, almost appearing flat on such a large scale.

In the end, A* with Manhattan Distance Heuristic is the most efficient algorithm as it explores the least nodes and maintains the smallest maximum queue size. MTH explores 96 nodes at depth 12, while MDH explores 36 nodes at the same depth. That's nearly 3x fewer nodes. This proves the MDH algorithm is more optimal and scalable for puzzles like the 8-puzzle search with deeper solution depths.

Additional Examples

I also showed the traceback of two puzzles for the depth of 4 and the depth of 16 for the Manhattan Distance Heuristic, listed on the pages following the conclusion. These tracebacks show the complexity of the search process for different puzzle levels. It showcases the efficiency and performance of the MDH.

Conclusion

After observing the three different algorithms, Uniform Cost Search, A* with Misplaced Tile Heuristic, and A* with Manhattan Distance Heuristic, I can conclude that the Uniform Cost Search (UCS) algorithm is by far the least efficient approach of the three algorithms. It takes too much time and space to run, especially for more complex puzzles of deeper depths, as it expands exponentially larger. Compared to UCS, the Misplaced Tile Heuristic algorithm is significantly better in terms of saving space and time. In the end, however, Manhattan Distance is the best of the three. This project shows that the Manhattan Distance Heuristic is essential for solving complicated sliding puzzles like the 8-puzzle. It's overall the most efficient, meaning it is the most optimal out of the three algorithms. It consistently explored the fewest number of nodes and maintained the lowest max queue size throughout the depths of the puzzle.

The following text shown is a traceback of an Easy puzzle with A* Manhattan Distance Heuristic:

- Depth 2

Select an algorithm:

- 1) uniform cost search,
 - 2) misplaced tile heuristic, or
 - 3) the Manhattan distance heuristic
- 3

The best state to expand with a $g(n) = 0$ and $h(n) = 2$ is...

[1, 2, 0]

[4, 5, 3]

[7, 8, 6]

The best state to expand with a $g(n) = 1$ and $h(n) = 1$ is...

[1, 2, 3]

[4, 5, 0]

[7, 8, 6]

The best state to expand with a $g(n) = 2$ and $h(n) = 0$ is...

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

End of search!

Solution depth was 2

Number of nodes expanded: 2

Max queue size: 4

The following text shown is a traceback of a Hard puzzle with A* Manhattan Distance Heuristic:

- Depth: 16

8-Puzzle Solver

Type '1' to use a default puzzle, or '2' for a custom one

2

Enter your own custom puzzle

Enter your puzzle with spaces separating each number
and a 0 to represent the blank space.

Please only enter valid 8-puzzles!

Enter row 1 (Hit enter when done)1 6 7

Enter row 2 (Hit enter when done)5 0 3

Enter row 3 (Hit enter when done)4 8 2

Your puzzle is:

[1, 6, 7]

[5, 0, 3]

[4, 8, 2]

Select an algorithm:

1) uniform cost search,

2) misplaced tile heuristic, or

3) the manhattan distance heuristic

3

The best state to expand with a $g(n) = 0$ and $h(n) = 12$ is...

[1, 6, 7]

[5, 0, 3]

[4, 8, 2]

The best state to expand with a $g(n) = 1$ and $h(n) = 11$ is...

[1, 0, 7]

[5, 6, 3]

[4, 8, 2]

I deleted about 11 pages for the purpose of saving space in this report.

The best state to expand with a $g(n) = 15$ and $h(n) = 1$ is...

[1, 2, 3]

[4, 5, 6]

[7, 0, 8]

The best state to expand with a $g(n) = 16$ and $h(n) = 0$ is...

[1, 2, 3]

[4, 5, 6]

[7, 8, 0]

End of search!

Solution depth was 16

Number of nodes expanded: 94

Max queue size: 104

Code Shown Here in GitHub: https://github.com/kayliezhao/cs170_lab_1